



Data Pipeline & Data Observability

AICB-P1 · Ngày 10 · Bản phát cho học viên

Trần Quang Thiện

TrustedAI

VinUniversity · Phase 1 · Tuần 2 · 2026

HÃY SUY NGHĨ...

“Agent của bạn đọc data từ database công ty. Data đột nhiên sai — agent bắt đầu trả lời bậy. Bạn có biết không, hay đợi user báo?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Data Pipeline — Nền Tảng Của Mọi AI Product
2. Ingestion — Kéo Data Về Mà Không Mất Bản Ghi
3. Transform — Clean & Chuẩn Hoá Data
4. Data Quality — 6 Dimensions
5. Data Observability
6. Từ Phát Hiện Đến Chính Sách
7. Orchestration — Đưa Pipeline Vào Vận Hành

Bốn phần lý thuyết

- ETL / ELT · batch / streaming
- Ingestion · cleaning · data contract
- Data quality: 6 dimensions
- Observability: 5 pillars · freshness SLA · alert

Ba hoạt động xen giữa

- **A** — tự tìm lỗi trong data thật
- **B** — drift detection nói thật được bao nhiêu
- **Thảo luận** — sự cố P1 lúc 09:12

Làm trên data thật

Cả ba hoạt động chạy trên **data taxi NYC thật** — không phải data giả. Lỗi trong đó là lỗi có sẵn, bạn tự tìm ra.

Ngày 08 — RAG

Nguồn tri thức đáng tin, retrieve đúng đoạn.

“Refund 7 ngày” lấy từ đâu ra?

Ngày 09 — Multi-agent

Supervisor + workers, route và trace được.

Worker nào đã trả lời câu này?

Ngày 10 — Data + observe

Cập nhật và monitor nguồn tri thức đó.

Data vào có còn đúng không — và ai biết trước user?

Cùng một artifact

Hai ngày trước bạn xây agent đọc knowledge base. Hôm nay bạn chịu trách nhiệm cho **cái đồ vào knowledge base đó**.

- 1. Data Pipeline — Nền Tảng Của Mọi AI Product**
2. Ingestion — Kéo Data Về Mà Không Mất Bản Ghi
3. Transform — Clean & Chuẩn Hoá Data
4. Data Quality — 6 Dimensions
5. Data Observability
6. Từ Phát Hiện Đến Chính Sách
7. Orchestration — Đưa Pipeline Vào Vận Hành

Data Pipeline — Nền Tảng Của Mọi AI Product

Đọc đúng tầng bị lỗi trước khi sửa prompt hay đổi model

- **60–80% thời gian** của một AI project thực tế là data work — không phải model
- Một RAG agent xuất sắc vẫn **hallucinate** nếu vector store được nạp data không clean
- **Garbage in** → **garbage out** không phải khẩu hiệu: quality của output không thể cao hơn quality của input
- Observability là cơ chế phát hiện data sai **trước khi user phàn nàn**

Thực tế một dự án AI:

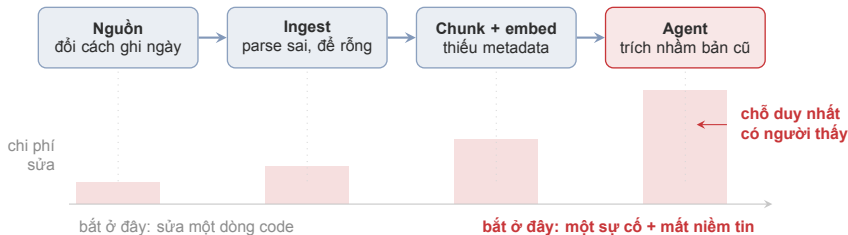
20% — xây model / agent

80% — thu thập, clean data, dựng pipeline, monitor

Câu hỏi xuyên suốt hôm nay

Khi agent trả lời sai — bạn mở cái gì ra xem **đầu tiên**?

Data cascade — một quyết định nhỏ ở đầu nguồn, trôi xuôi qua từng bước, mỗi bước làm hậu quả lớn thêm — và **chỉ lộ ra ở cuối**.



Data không có compiler. Không bước nào ở giữa báo đồ — nên nó trôi hết cả bốn bước.

Điều quan trọng nhất

Mỗi cascade đều bắt đầu bằng một quyết định **hợp lý tại chỗ**, không phải do ai cầu thả. Nên bạn **không thoát khỏi nó bằng cách kêu gọi cẩn thận hơn** — bạn thoát bằng cách **gắn cảm biến sớm hơn**.

“Đúng fact, nhưng là fact cũ”

→ freshness · publish step · cache vector chưa swap

“Lẫn lộn giữa hai phiên bản tài liệu”

→ dedupe · version metadata · chunk boundary

“Tự bịa, nhưng nghe rất hợp lý”

→ thiếu grounding + retrieval gap — **vẫn kiểm data trước**

“Chỉ sai với một loại ticket”

→ khoanh vùng theo lineage: nguồn OCR hoặc API nào?

“Đang tốt, sau deploy thì hỏng”

→ schema / parser đổi — so sánh distribution trước và sau

Nguyên tắc: đo freshness và volume **trước** khi mở notebook fine-tune.

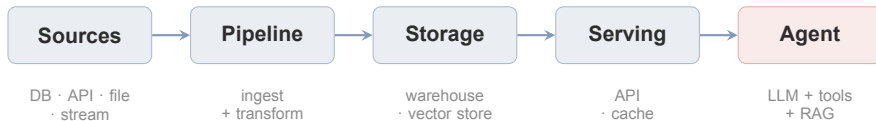
Vai trò	Trách nhiệm chính	Artifact bàn giao
AI / Applied	data contract cho corpus của agent · eval before/after	data_contract.md · golden questions
Data Eng	ingest ổn định · modeling · pipeline SLA	pipeline_architecture.md · lineage
SRE / Platform	alert · on-call runbook · quota và secret	runbook.md · dashboards
Product / SME	định nghĩa thế nào là “đúng” · duyệt version	link tới source of truth

SME = subject matter expert — người nắm nghiệp vụ, biết chắc thế nào là đúng.
SLA = mức cam kết về thời gian, thoải thuận trước với người dùng.

Ranh giới quan trọng

Mỗi nguồn data phải có đúng một owner có tên. Một người đóng nhiều vai thì được; một nguồn không ai đứng tên thì không.

Data Pipeline — Chuỗi bước tự động **thu thập, xử lý** và **phân phối** data từ nguồn đến nơi agent đọc

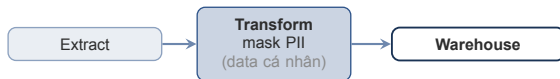


Chú ý

Bốn mũi tên này là bốn chỗ data có thể hỏng mà **agent vẫn chạy bình thường**. Không có chỗ nào tự báo lỗi.

Cả hai đều gồm đúng ba việc — **Extract** (lấy data ra khỏi nguồn) · **Transform** (biến đổi, làm sạch) · **Load** (nạp vào kho). Khác nhau **chỉ ở thứ tự**.

ETL — transform **trước** khi vào warehouse



Dừng khi bạn KHÔNG ĐƯỢC PHÉP giữ raw.

VD: log có số CMND — luật buộc mask trước khi lưu.

ELT — ghi raw trước, transform **sau** bằng SQL



Dừng khi bạn CHƯA CHẮC sau này xử lý kiểu gì.

VD: PDF policy — 3 tháng sau muốn cắt chunk kiểu khác, chạy lại SQL là xong.

Chọn bằng một câu hỏi

Bạn có được phép giữ raw không? Không → **ETL**, hết lựa chọn. Được → **ELT** — vì hôm nay bạn chưa biết ba tháng nữa sẽ muốn transform kiểu gì. Giữ raw là giữ quyền đổi ý.



Batch — rẻ, dễ chạy lại. Dùng cho training data, report định kỳ.

Streaming — đắt, khó debug. Dùng cho fraud detection, context realtime.

Chọn bằng câu hỏi này

Data trễ 6 tiếng thì agent trả lời sai tới mức nào? Trả lời được câu đó mới chọn được — không phải cứ realtime là tốt hơn.

HOẠT ĐỘNG TRÊN LỚP

6 phút

Bốn tình huống dưới đây. Với mỗi cái, chọn **ETL hay ELT** và **batch hay streaming**, kèm một dòng lý do.

1. **Mask PII trong log** trước khi log vào data lake
2. **Đồng bộ đêm** file `policy/refund-v4.pdf` vào knowledge base
3. **Webhook ticket P1** cần lên dashboard ngay lập tức
4. **File CSV hàng tháng** `tickets/2026-02-incidents.csv`

Cân nhắc theo bốn trục: **latency · governance · cost · khả năng debug.**

Trả lời những câu dưới đây, rồi đăng lên Discord.

1. Với mỗi tình huống trong bốn tình huống: bạn chọn **ETL hay ELT? Batch hay streaming?**
2. Mỗi lựa chọn, viết **một dòng lý do** — theo bốn trục: latency · governance · cost · khả năng debug.
3. Tình huống nào bạn thấy **khó chọn nhất**, và vướng ở chỗ nào?
4. Với lựa chọn của bạn, **bạn đang đánh đổi cái gì?** (đây mới là phần được chấm)

1. Data Pipeline — Nền Tảng Của Mọi AI Product
- 2. Ingestion — Kéo Data Về Mà Không Mất Bản Ghi**
3. Transform — Clean & Chuẩn Hoá Data
4. Data Quality — 6 Dimensions
5. Data Observability
6. Từ Phát Hiện Đến Chính Sách
7. Orchestration — Đưa Pipeline Vào Vận Hành

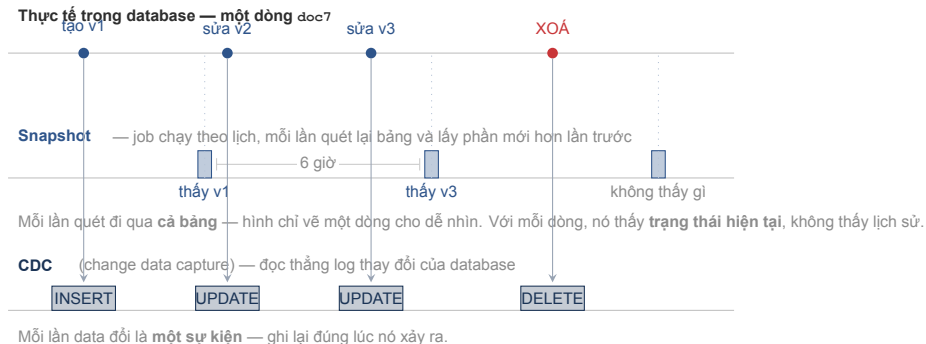
Ingestion — Kéo Data Về Mà Không Mất Bản Ghi

Mỗi loại nguồn hổng theo một kiểu riêng — và cần một cách đo riêng

Nguồn	Hồng kiểu gì	Đo bằng cái gì
PostgreSQL	timeout · schema drift · job kéo về 0 dòng	freshness · số dòng kéo về mỗi lần chạy
External API	429 rate limit · auth hết hạn · pagination lệch	volume · checkpoint có nhích không
PDF / HTML	OCR sai · encoding · thiếu metadata	% dòng bị flag review
Webhook / stream	backpressure · trùng event	queue depth · số bản ghi vào DLQ

Template dùng được ngay

Mỗi nguồn viết đúng một dòng: `source` → `method` → `check` → `owner`. Nguồn nào không điền nổi cột `owner` là nguồn sẽ gây sự cố.



Khác nhau ở chỗ nào

Snapshot chụp lại **trạng thái** theo lịch — như ảnh chụp mỗi 6 giờ. **CDC** ghi lại **từng thay đổi** — như video. Chọn cái nào là việc của team quản lý database nguồn.

429 = “bạn gọi nhiều quá”. Bên kia thường kèm header `Retry-After` — đọc nó trước khi tự đoán.

Backoff — mỗi lần thất bại thì chờ gấp đôi, không dồn dập vào API đang quá tải



KHÔNG jitter — 3 worker cùng chờ đúng 2s



CÓ jitter — cộng thêm một chút ngẫu nhiên



Sai kinh điển

Retry vô hạn, không backoff — pipeline **tự DDoS** chính API nguồn. Bên kia chặn bạn, và họ đúng.

Data **vẫn đang đổi** trong lúc bạn đọc — đó là lý do cách đánh số trang lại quan trọng.

OFFSET — “cho tôi từ dòng thứ 3 trở đi”

Trang 1:

101	102	103
-----	-----	-----

 đọc xong 3 dòng

CHÈN:

100

 một bản ghi mới chèn vào **đầu** danh sách — mọi thứ dịch xuống một ô

Trang 2:

103	104	105
-----	-----	-----

103 đọc lần hai — còn 100 thì không bao giờ được đọc

CURSOR — “lần trước tôi dừng ở 103, cho tôi phần sau nó”

Trang 1:

101	102	103
-----	-----	-----

nhớ mốc: `id > 103`

Trang 2:

104	105
-----	-----

chèn bao nhiêu cũng không lệch

Checkpoint idempotent: lưu lại chính cái mốc đó (`last_synced_id`) — chạy lại thì đi tiếp, không kéo lại toàn bộ lịch sử.

Một trang lỗi $\neq$ cả job lỗi

Ghi trang hỏng vào **DLQ** rồi chạy lại đúng nó — thay vì đánh sập cả job vì một trang.

Content hash $\neq$ logical version

`sha256` của bytes trả lời **“file có đổi không?”** — để bỏ qua file không đổi, khỏi embed lại.

`policy_v4` trả lời câu khác hẳn: **“đây là bản nào?”**

Thiếu cái đầu: không biết file nào đã đổi → **lần chạy nào cũng embed lại tất cả**. Thiếu cái sau: agent **không lọc được theo version**.

OCR có điểm tin cậy

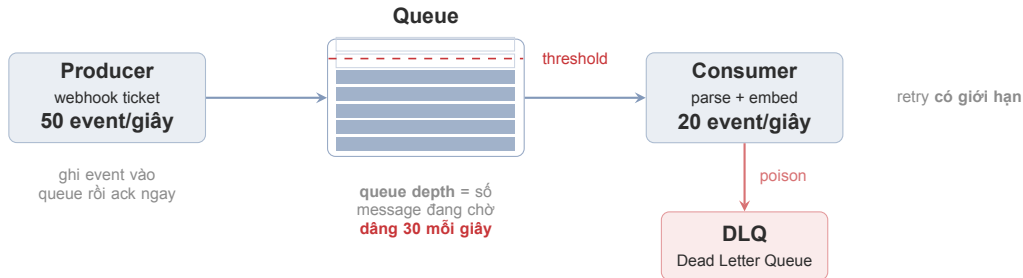
OCR — công nghệ đọc chữ từ ảnh scan.

Thấp thì flag lại cho người review **trước khi** publish — không phải sau khi user phàn nàn.

Cắt chunk theo heading

Cắt theo số ký tự làm agent trả lời “nửa điều khoản” — vẫn có trích dẫn, vẫn sai nghĩa.

Nói về ngày 08: metadata bạn gắn ở bước này chính là thứ *filter* của vector store sẽ dùng. Không gắn ở đây thì sau này không lọc được.



- **Queue phình** — consumer chậm hơn producer. Đây là **triệu chứng**, không phải nguyên nhân: phải hỏi consumer chậm vì cái gì.
- **Backpressure** — cơ chế **ghì ngược lên producer**: queue gần đầy thì báo ngược “chậm lại”. Không có nó, queue phình tới khi hết bộ nhớ rồi mất data.
- **Poison message** — bản ghi retry mấy lần cũng lỗi. Không tách ra thì nó chặn cả queue phía sau.
- **DLQ (Dead Letter Queue)** — queue riêng chứa những message đó. Nó là **điểm quan sát, không phải chỗ vứt rác**: event hỏng còn nằm đó cho bạn xem, thay vì biến mất im lặng.

Ví dụ trên hình: ticket đổ về nhanh gấp 2,5 lần tốc độ xử lý → sau một giờ có **hơn 100.000 message** nằm chờ, agent đọc knowledge base cũ dần mà không ai báo. **Do queue depth và số vào DLQ** là thấy trước khi user thấy.

CHECKPOINT — Tự kiểm tra

Pipeline báo “thành công” mỗi đêm nhưng data không mới. Nghi cái gì đầu tiên?

✓ Job chạy xong không lỗi, nhưng **kéo về 0 dòng**. Với hệ thống, “không có dòng mới” trông y hệt “thành công” — nên phải đo **số dòng kéo về**, không chỉ đo job có chạy hay không.

Vì sao cursor an toàn hơn offset khi pagination?

✓ Vì data đổi trong lúc đọc. Có bản ghi chen vào giữa thì offset làm lệch mọi trang sau — đọc trùng hoặc bỏ sót, mà không có lỗi nào báo ra.

Queue phình liên tục. Tăng số consumer có phải là fix không?

✓ Là giảm đau, không phải fix. Cần biết consumer chậm vì sao: xử lý nặng, phụ thuộc bên ngoài chậm, hay đang retry một poison message mãi không xong.

Trả lời được cả ba thì đi tiếp phần Transform.

1. Data Pipeline — Nền Tảng Của Mọi AI Product
2. Ingestion — Kéo Data Về Mà Không Mất Bản Ghi
- 3. Transform — Clean & Chuẩn Hoá Data**
4. Data Quality — 6 Dimensions
5. Data Observability
6. Từ Phát Hiện Đến Chính Sách
7. Orchestration — Đưa Pipeline Vào Vận Hành

Transform — Clean & Chuẩn Hoá Data

Biến raw data thành thứ agent dùng được mà không phải đoán

RAW

id	title	date	content	vấn đề
12	Refund policy	07/02/26	Refund 7 ngày	sai định dạng
12	Refund policy	2026-02-07	Refund 7 ngày	trùng khoá
44	Access approve	NULL	Quy trình duyệt	thiếu ngày
58	P1 SLA	2026/02/10	SLA 2h ?	lỗi encoding

CLEANED

id	ver	hiệu lực	status	ghi chú
12	v4	2026-02-07	active	giữ bản parse được
44	v2	đã flag	review	chưa embed, đợi SME
58	v1	2026-02-10	active	đã chuẩn hoá NFC

parse ngày · chuẩn hoá unicode NFC (gộp ký tự có dấu về một cách viết) · dedupe theo khoá · check schema

Nguyên tắc

Transform là **code + contract**, không phải sửa tay trong Excel năm phút trước khi demo. Sửa tay thì lần chạy sau lỗi lại y nguyên.

Năm kiểu lỗi hay gặp:

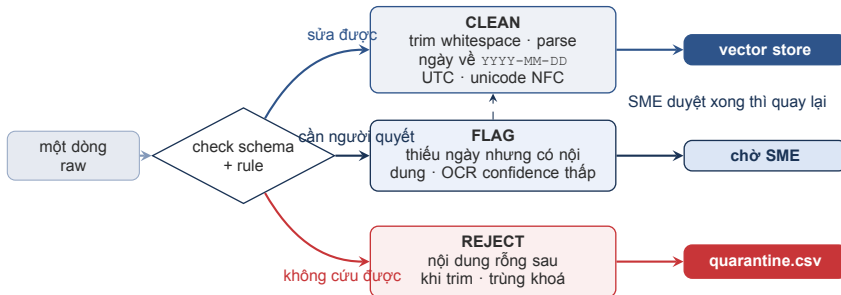
- **Thiếu giá trị** — `NULL`, chuỗi rỗng, “N/A”, hoặc số 0 giả làm “chưa có”
- **Outlier** — một dòng bất thường đủ để kéo lệch cả embedding
- **Trùng lặp** — cùng bản ghi vào nhiều lần; dedupe theo hash nội dung hoặc khoá tự nhiên
- **Sai định dạng** — 31/12/2024 và 2024-12-31 trong cùng một cột
- **Lỗi encoding** — UTF-8 lẫn Latin-1; tiếng Việt có dấu là chỗ lộ ra đầu tiên

Chuẩn hoá text trước khi embed:

- **Unicode NFC** — bắt buộc với tiếng Việt, nếu không thì hai chuỗi trông giống hệt nhau vẫn không khớp
- **Dọn text** — trim khoảng trắng thừa, bỏ tag HTML nhưng giữ cấu trúc heading

Check schema: reject bản ghi sai schema ngay ở bước nhận data — thay vì để nó lọt vào rồi đi tìm nguyên nhân sau ba tuần.

Với giá trị thiếu: bỏ, điền, hay flag — chọn cách nào phải ghi vào contract.



Điều kiện bắt buộc

Mọi bản ghi bị reject hoặc flag phải được **ghi lại kèm run_id** vào `quarantine.csv`.
Xoá mà không log nghĩa là bạn vừa mất data mà không ai biết.

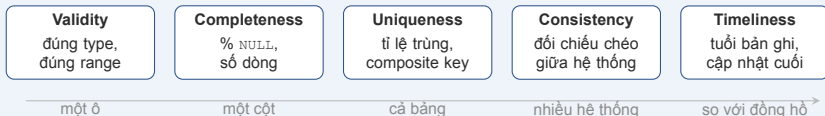
1. Data Pipeline — Nền Tảng Của Mọi AI Product
2. Ingestion — Kéo Data Về Mà Không Mất Bản Ghi
3. Transform — Clean & Chuẩn Hoá Data
- 4. Data Quality — 6 Dimensions**
5. Data Observability
6. Từ Phát Hiện Đến Chính Sách
7. Orchestration — Đưa Pipeline Vào Vận Hành

Data Quality — 6 Dimensions

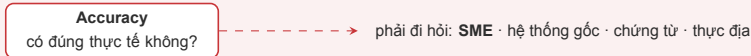
Đo chất lượng data bằng code, trước khi data đi vào agent

04

MÁY TỰ CHECK ĐƯỢC — chỉ cần nhìn vào chính bảng đó



CẦN SOURCE OF TRUTH BÊN NGOÀI — không check nào tự trả lời được



Chỗ hay nhầm

Validity và Accuracy khác nhau. `fare_amount = 3000` **hợp lệ** (là số, dương) nhưng có thể **không đúng**. Check tự động bắt được cái đầu; cái sau cần source of truth bên ngoài.

data_contract.md · NYC Yellow Taxi · v3

2 · Danh mục giá trị hợp lệ

payment_type: 1 thẻ · 2 tiền mặt · 3 miễn phí
4 tranh chấp · 5 không rõ · 6 chuyển bị hủy

3 · Quy tắc nghiệp vụ

“Tip chỉ được ghi khi khách trả bằng thẻ. Tip tiền mặt không bao giờ vào hệ thống.”

“Số tiền âm là hợp lệ khi chuyển là hoàn tiền.”

4 · Những gì contract KHÔNG quy định

“Không có trần cho fare_amount.”

“Không có quy tắc total_amount phải bằng tổng các khoản lẻ.”

Mục 2 cho bạn cái gì

Viết được check tự động **ngay hôm nay** — giá trị ngoài danh mục là sai, không cần bàn.

Mục 3 cứu bạn khỏi cái gì

Chặn nhầm data đúng. Không có nó, bạn sẽ viết rule chặn `tip = 0` và chặn tiền âm — cả hai đều hợp lệ.

Mục 4 là mục quan trọng nhất

Nó cho bạn quyền trả lời “**KHÔNG THỂ BIẾT**” — thay vì đoán bừa rồi tự tin sai.

Contract không **mô tả** data — nó dùng để **phán quyết**. Thấy con số lạ, tra xem nó rơi vào mục 2, 3 hay 4.

HOẠT ĐỘNG TRÊN LỚP

30 phút · làm một mình

Data: 5.000 chuyến taxi vàng New York, tháng 01/2019. **Data thật, lỗi thật** — không ai cài lỗi vào đây.

Bạn có trong tay:

- `data_contract.md` — định nghĩa thế nào là một dòng “bình thường”
- `explore.py` — viết code của bạn trong hàm `your_turn()`. Có sẵn một phát hiện làm mẫu + 5 helper: `describe` `group` `cross` `derive` `show`
- `worksheet.html` — chỗ ghi phát hiện, tự lưu trong trình duyệt

Mỗi phát hiện phải ghi đủ 5 dòng: thấy gì · tìm bằng cách nào · dimension nào · **phán quyết** · cần gì để chắc chắn.

Ba phán quyết được phép chọn: **SAI** (vi phạm điều viết rõ trong contract) · **ĐÚNG** (trông lạ, nhưng contract giải thích được) · **KHÔNG THỂ BIẾT** (contract không nói gì — ghi rõ cần nguồn nào)

Trả lời những câu dưới đây, rồi đăng lên Discord.

1. **Ba phát hiện** của bạn — mỗi cái đủ năm dòng: thấy gì · tìm bằng cách nào · dimension nào · phán quyết · cần gì để chắc chắn.
2. Phát hiện nào bạn phán **ĐÚNG** (trông lạ nhưng contract giải thích được)? Contract mục nào cho bạn kết luận đó?
3. Phát hiện nào rơi vào **KHÔNG THỂ BIẾT**? Bạn cần **hỏi nguồn nào** để chắc chắn?
4. Với từng phát hiện: **nếu điều này sai thật, ai là người biết chắc?**

Expectation — Một khẳng định về data mà máy check được — chạy được, log được, đưa vào CI được

1. **Profile** — GX đọc data và gợi ý expectation
2. **Viết suite** — not-null, range giá trị, danh mục, regex
3. **Validate** trước khi data đi tiếp
4. **Report** — HTML data docs tự sinh
5. **Chặn** — pipeline dừng khi suite fail

API đã đổi. Bản 1.x khác hẳn 0.x. Pin đúng version:

```
great-expectations==1.19.1
```

```
import great_expectations as gx, pandas as pd

df = pd.read_csv("taxi_reference.csv")
ctx = gx.get_context(mode="ephemeral")
batch = (ctx.data_sources.add_pandas("taxi")
        .add_dataframe_asset(name="trips")
        .add_batch_definition_whole_dataframe("all")
        .get_batch(batch_parameters={"dataframe": df}))

suite = ctx.suites.add(gx.ExpectationSuite(name="s"))
suite.add_expectation(gx.expectations.
    ExpectColumnValuesToBeBetween(
        column="passenger_count", min_value=1, max_value=6))
print(batch.validate(suite).success)
```

CHECKPOINT — Tự kiểm tra

Suite expectation pass hết. Data đã đúng chưa?

✓ Chưa. Pass nghĩa là data **không vi phạm những gì bạn nghĩ ra**. Accuracy cần đối chiếu source of truth bên ngoài — không check nào tự làm được.

Vì sao `fare_amount` âm không nên để pipeline dừng?

✓ Vì nó hợp lệ — đó là bản ghi hoàn tiền. Rule chặn nó là **rule chưa khớp nghiệp vụ**, không phải data sai. Chặn kiểu đó làm cả nhóm mất niềm tin vào alert.

Bạn viết check cho dimension nào trước, nếu chỉ có thời gian cho một dimension?

✓ Validity — rẻ nhất, tự động hoàn toàn, và chặn được đúng loại lỗi thiết bị sinh ra nhiều nhất. Nhưng phải biết rõ nó **không** bảo vệ bạn khỏi accuracy.

Phần tiếp theo: làm sao biết data hỏng khi không ai chạy check.

1. Data Pipeline — Nền Tảng Của Mọi AI Product
2. Ingestion — Kéo Data Về Mà Không Mất Bản Ghi
3. Transform — Clean & Chuẩn Hoá Data
4. Data Quality — 6 Dimensions
- 5. Data Observability**
6. Từ Phát Hiện Đến Chính Sách
7. Orchestration — Đưa Pipeline Vào Vận Hành

Data Observability

Biết data hỏng trước khi user biết — và biết hỏng ở đâu

05

Bốn pillar đầu — đo tại một điểm, chỉ nói “đang có sự cố”

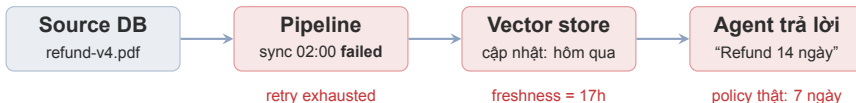


Lineage — đường đi nối các điểm đo lại với nhau

Công cụ đo những thứ này: Great Expectations (quality check) · Evidently (drift)

Vì sao lineage khác hẳn bốn cái kia

Freshness 17h cho bạn biết **đang có sự cố**. Chỉ lineage nói được **sự cố nằm ở bước nào** — ở đây là ingest lúc 02:00, không phải ở model.



Không có observability

Lần publish thành công cuối lúc 16:00 hôm trước.
Sync 02:00 fail im lặng. **09:12** user đầu tiên phàn nàn — data đã cũ **17 tiếng** và không ai biết.

Có observability

Freshness vượt threshold lúc **02:30**. On-call nhận alert, chạy lại ingest, xong trước giờ làm việc.
Không user nào thấy câu trả lời sai.

Điểm mấu chốt

Pipeline **không hề báo lỗi**. Job kết thúc, log không báo gì. Thứ duy nhất sai là **không có gì mới** — và “không có gì mới” trông y hệt “không có gì cần làm”.

SLA = lời hứa với người dùng, có con số và có thời hạn. Không phải mục tiêu nội bộ — vỡ là có người phải chịu trách nhiệm.

Không viết: “pipeline chạy mỗi 4 tiếng”

Viết: “policy refund phải xuất hiện trong câu trả lời của agent **trong vòng 4 giờ** kể từ khi PDF được ký”

Đo ở đâu

Tại thời điểm document **active** trên vector index — không phải lúc file rơi vào thư mục staging. Đo sai điểm thì SLA xanh mà user vẫn thấy data cũ.

Nguồn khác nhau, SLA khác nhau

PDF policy tĩnh và luồng ticket không dùng chung một threshold.

Nói cho user biết

Hiển thị dòng “Dữ liệu cập nhật tới: ...” — giảm rất nhiều kỳ vọng sai.

Hai mức alert

Đi hết nửa thời gian — 2 giờ → warning

Vào ticket queue. Còn kịp xử lý trong giờ hành chính.

Hết 4 giờ → page

Page thẳng cho người on-call, kể cả 3 giờ sáng. Đã vi phạm cam kết với người dùng.

Gọi người vì user đang chịu ảnh hưởng
— không phải vì job màu đỏ.

Null rate	<code>effective_date</code> null tăng vọt sau khi đổi API — schema vẫn đúng, số dòng vẫn đủ
Độ dài nội dung	Chunk ngắn bất thường thường là dấu hiệu parser vừa hỏng
Ngôn ngữ / nhiễu	Text lẫn ngôn ngữ lạ hoặc rác OCR tăng đột biến
Ràng buộc chéo cột	<code>end < start · version</code> mới nhỏ hơn version cũ — từng cột đều hợp lệ, ghép lại thì vô lý
Thời gian tương lai	<code>effective_date</code> vượt quá hiện tại — lệch múi giờ hoặc lỗi parse

Vì sao nhóm này khó

Không cái nào làm pipeline fail. Chúng chỉ làm câu trả lời của agent **tệ dần** — kiểu hỏng khó phát hiện nhất vì không có một thời điểm hỏng rõ ràng để lần theo.

HOẠT ĐỘNG TRÊN LỚP

30 phút · làm một mình

Hai file: `taxi_reference.csv` — data kỳ trước, lấy làm **mốc so sánh** (“xưa nay data trông như thế này”). `taxi_current.csv` — data mới về, cần kiểm tra.

Drift detection = so phân bố giá trị của hai file, xem đã đổi chưa.

1. Chạy `python3 drift.py`. Cột nào bị đánh dấu **LỆCH**?
2. **Đọc** `drift.py` — cả file 40 dòng, không dùng library ngoài. Viết một câu: **nó thật sự đo cái gì?**
3. Chạy `python3 drift.py fare_amount`, xem chi tiết từng khoảng giá trị.
4. Ở hoạt động A bạn đã tìm được vài thứ hổng thật. **Drift có bắt được cái nào không?**

Câu hỏi kết: **cột duy nhất bị đánh dấu — nó có thật sự hổng không?**

Trả lời những câu dưới đây, rồi đăng lên Discord.

1. Cột nào bị đánh dấu **LỆCH**? Điểm lệch bao nhiêu?
2. Đọc `drift.py` — viết **một câu**: nó thật sự đo cái gì?
3. Cột đó có **thật sự hỏng** không? Bạn dựa vào đâu để kết luận?
4. Những lỗi bạn tìm được ở hoạt động A — `drift.py` bắt được **cái nào**? Vì sao?

Bạn vừa viết được thứ này trong 40 dòng. Khi lên production thì dùng library có sẵn:

- Tự chọn phép thống kê theo type của cột (số, categorical, text)
- Report HTML có biểu đồ từng cột
- Threshold cấu hình được, xuất được JSON để đưa vào alert
- Chạy được trong pipeline như một task

`Report([DataDriftPreset()])` rồi `save_html()` —
ba dòng.

Library không cứu bạn khỏi bài học ở hoạt động B

Evidently vẫn `alert passenger_count`. Nó cũng vẫn im lặng trước các dòng `total_amount` lệch.

Công cụ tốt hơn **không sửa được** việc reference bị chọn sai và câu hỏi bị đặt sai.

Citation freshness

Tuổi trung bình và p95 (95% trường hợp nằm dưới mức này) của chunk được agent trích dẫn — **đây là freshness user thật sự nhìn thấy**

Grounding rate

% câu trả lời có trích dẫn nguồn hợp lệ

Retrieval hit@k

Trên một golden set câu hỏi nhỏ, cố định — chạy mỗi lần publish

Latency ingest → publish

p95, tách riêng khỏi latency của LLM; nếu không thì mọi thứ chậm đều bị đổ cho model

Thứ tự đúng

Khi các chỉ số này xấu đi, quay lại 5 pillars **trước khi** tin rằng “model bị lỗi”. Model hiếm khi tự dừng kém đi sau một đêm — data thì có.



“Khi agent trả lời cũ, đừng debug model trước. Freshness → Volume → Schema → Lineage → nguyên nhân.”

— Thứ tự triage của ngày hôm nay

HOẠT ĐỘNG TRÊN LỚP

10 phút · thảo luận nhanh

Tình huống: 09:12, user phàn nàn agent đang trả lời theo policy cũ. Bạn mở được cả sáu thứ dưới đây.

Việc của bạn: chọn **4 thẻ**, xếp theo **thứ tự bạn sẽ mở**, và ghi **con số bạn đi tìm** ở mỗi thẻ kèm **ngưỡng để so** — ví dụ “`freshness_hours` so với ngưỡng 4h”.

A · Freshness

`freshness_hours` trên bảng
`kb_documents`

B · Volume

`rows_ingested` so với 7
ngày trước

C · Eval của model

điểm eval mới nhất của LLM

D · Schema contract

parser PDF có drift không

E · Log ingest thô

2.000 dòng log của job đêm
qua

F · Lineage

đường đi của
`refund-v4.pdf`

Hai thẻ là bắt. Chọn nhầm thì vẫn ra câu trả lời — chỉ là mất thêm 20 phút. Nói được vì **sao bỏ chúng** mới là phần điểm.

Gợi ý loại thẻ: cái **không tự đổi sau một đêm** · cái **chưa khoanh vùng thì không đọc nổi**.

Trả lời những câu dưới đây, rồi đăng lên Discord.

1. **Bốn thẻ** bạn chọn, **theo thứ tự** sẽ mở.
2. Mỗi thẻ: **con số** bạn đi tìm và **ngưỡng để so**.
3. **Hai thẻ nào là bẫy?** Vì sao bạn bỏ chúng? (phần điểm nằm ở đây)
4. Sau khi tìm ra nguyên nhân — **làm sao biết là đã fix xong?**

0–5 phút

Freshness + `last_success_run`. Có trễ SLA không?

5–12 phút

Volume + error rate từng bước. Bước nào **im lặng bất thường** hoặc vọt lên?

12–20 phút

Schema / contract, rồi lần lineage ngược về bảng hoặc file nguồn.

Hết 20 phút chưa ra nguyên nhân?

Chuyển sang giảm thiệt hại. Rollback bản publish trước, hoặc bật banner “dữ liệu đang cập nhật”.

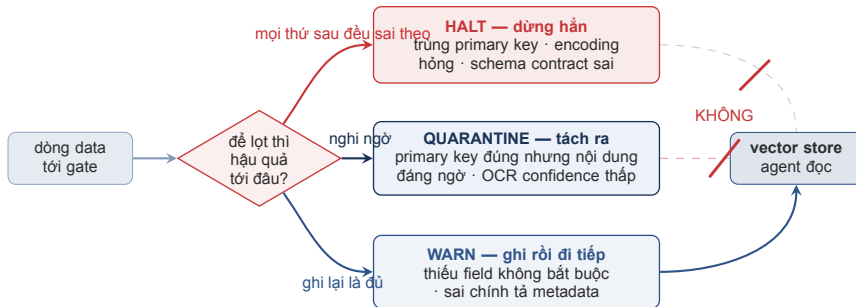
User đang chờ. Tìm nguyên nhân là việc quan trọng — nhưng không quan trọng bằng việc chặn thiệt hại lại.

Ghi lại mọi metric đã xem và kết luận từng bước — kể cả bước dẫn tới ngõ cụt. Lần sau đỡ mất 20 phút đó.

1. Data Pipeline — Nền Tảng Của Mọi AI Product
2. Ingestion — Kéo Data Về Mà Không Mất Bản Ghi
3. Transform — Clean & Chuẩn Hoá Data
4. Data Quality — 6 Dimensions
5. Data Observability
- 6. Từ Phát Hiện Đến Chính Sách**
7. Orchestration — Đưa Pipeline Vào Vận Hành

Từ Phát Hiện Đến Chính Sách

Biết data hỏng là một chuyện — quyết định làm gì với nó là chuyện khác



Xếp mức nào là hỏi **cần làm**

gì và có chờ được không — không phải hỏi lỗi nghe đáng sợ tới đâu.

Chính sách phải viết ra

Ghi rõ trong `data_contract.md` mức nào ứng với lỗi nào. **Bảng quarantine không được để agent đọc** — nếu không, thứ bạn vừa tách ra vẫn đi thẳng vào câu trả lời.

HOẠT ĐỘNG TRÊN LỚP

25 phút · làm một mình

`validate.py` có sẵn **một** rule mẫu. Viết thêm **bốn** rule, dựa trên những gì bạn tự tìm được ở hoạt động A. Mỗi rule là một dòng — **không cần viết code**: ("`tên_cột`", "`phép so sánh`", `giá_trị`)
với `in` · `not_null` · `min` · `max` · `equals`

```
RULES = [ ("rate_code_id", "in", ["1","2","3","4","5","6"]) ]
```

```
$ python3 validate.py
```

```
LUẬT rate_code_id in ['1'...'6'] → chặn 1 dòng (0.0%)
```

```
ví dụ: 2019-01-22 rate=99 pay=2 fare=2.5
```

Dòng cuối — **những dòng bị chặn có điểm gì chung** — mới là về đáng đọc.

Với từng rule, trả lời bằng contract:

1. Những dòng bị chặn có **thật sự hỏng** không, hay contract cho phép chúng?
2. Bao nhiêu trong số bị chặn là lỗi thật? (tỉ lệ đó gọi là **precision**)
3. Rule này nên là **halt** / **quarantine** / **warn**? Rule nào bạn không dám để halt?

Trả lời những câu dưới đây, rồi đăng lên Discord.

1. **Bốn rule** bạn viết — và mỗi rule **chặn bao nhiêu dòng**?
2. Những dòng bị chặn **có điểm gì chung**? (đọc dòng thứ hai trong output)
3. Trong số bị chặn, bao nhiêu là **lỗi thật**? (precision)
4. Rule nào là **data sai**, rule nào là **bạn hiểu sai nghiệp vụ**?
5. Xếp mỗi rule: **halt / quarantine / warn**. Rule nào bạn không dám để halt — vì sao?

1. Data Pipeline — Nền Tảng Của Mọi AI Product
2. Ingestion — Kéo Data Về Mà Không Mất Bản Ghi
3. Transform — Clean & Chuẩn Hoá Data
4. Data Quality — 6 Dimensions
5. Data Observability
6. Từ Phát Hiện Đến Chính Sách
- 7. Orchestration — Đưa Pipeline Vào Vận Hành**

Orchestration — Đưa Pipeline Vào Vận Hành

Chạy được một lần là bài tập. Chạy lại an toàn mới là sản phẩm.

Trigger — cron, event, hay đợi task trước chạy xong?

Retry policy — mấy lần, backoff ra sao, và **retry** lại từ task lỗi hay từ đầu?

Data awareness — có sensor đợi file hoặc partition xuất hiện không?

SLA / alert — thời gian chạy task, queue depth, lịch chạy bị lỗi

Observability — log từng task + gắn `run_id` vào lineage

Backfill — chạy lại một khoảng ngày **mà không phá production**

Prefect

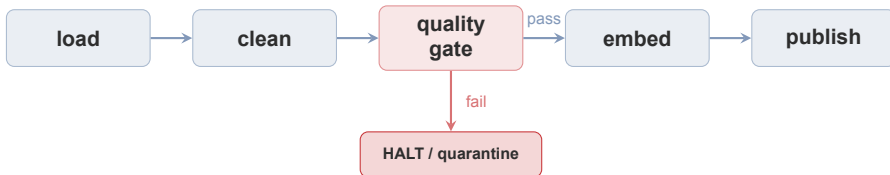
Flow là hàm Python bình thường. Ít boilerplate nhất — **bắt đầu ở đây**.

Dagster

Mô hình hoá **data asset**, không phải task. Lineage có sẵn, không cần dựng thêm.

Airflow

Phổ biến nhất, hệ sinh thái lớn nhất, cũng nặng nhất khi setup.



Vì sao gate đứng trước embed?

Embed tốn tiền và khó rút lại. Chặn ở đây rẻ hơn nhiều so với dọn vector store sau đó.

Câu hỏi khó hơn:

Drift tăng nhưng quality gate vẫn pass — hệ thống nên **chặn** hay chỉ **báo**? Ba hoạt động sáng nay là để bạn tự trả lời được câu này.

Upsert theo natural key

(doc_id, version) — không sinh UUID mới mỗi lần chạy. Chạy lại thì ghi đè đúng bản ghi cũ, không thêm bản mới.

Staging rồi đổi alias

Nạp vào collection mới, xong thì trở alias sang. Agent không bao giờ đọc phải collection đang nạp dở.

Lưu ý: Retry là cơ chế mặc định của mọi orchestrator — pipeline của bạn **sẽ** chạy lại, dù bạn có thiết kế cho việc đó hay không. Chạy lại để sửa data cũ mà không idempotent thì bạn **tự tạo ra bản trùng**: agent lấy được cả v3 lẫn v4, và trả lời hai kiểu khác nhau cho cùng một câu hỏi.

Symptom	User thấy gì? Agent trả lời sai theo kiểu nào?
Detection	Metric nào đã báo — hoặc chưa có metric nào báo , và đó chính là lỗi hổng
Diagnosis	Nguyên nhân gốc · bước pipeline nào · bảng hoặc file nào
Mitigation	Đã làm gì để dừng gây hại: rollback, chạy lại, tắt tính năng
Prevention	Expectation mới · alert mới · owner có tên

Dòng đáng giá nhất là Detection

Nếu không có metric nào báo trước user — thì việc cần làm sau sự cố không phải là sửa bug, mà là **thêm metric để đo**. Bug lần sau sẽ khác; chỗ không ai đo thì vẫn nguyên đó.

1. Data Pipeline — Nền Tảng Của Mọi AI Product
2. Ingestion — Kéo Data Về Mà Không Mất Bản Ghi
3. Transform — Clean & Chuẩn Hoá Data
4. Data Quality — 6 Dimensions
5. Data Observability
6. Từ Phát Hiện Đến Chính Sách
7. Orchestration — Đưa Pipeline Vào Vận Hành

Tổng Kết

Bốn điều mang về từ ba hoạt động sáng nay

08

1

Không có data contract thì mọi bất thường trông giống nhau — bạn sẽ chặn nhầm data đúng và bỏ lọt data sai cùng một lúc

2

Drift $\neq$ data sai. Drift chỉ nói distribution đã đổi, không nói ai đúng. Nó không thay được rule validate từng dòng — và ngược lại

3

“Job chạy xong” là chuyện của chương trình — “data đúng” là chuyện của data. Hai khẳng định khác nhau, và bạn chỉ đang đo cái thứ nhất

4

Khi agent trả lời sai: **freshness** $\rightarrow$ **volume** $\rightarrow$ **schema** $\rightarrow$ **lineage**, rồi mới đến model

Bài tiếp theo

Guardrails & AI Safety

*“Data đúng không có nghĩa là an toàn
— ngày mai là lớp bảo vệ ở mọi tầng”*

Bài tập về nhà

- Đọc: OWASP Top 10 for LLMs (owasp.org)
- Suy nghĩ: nếu ai đó cố tình bơm data độc vào nguồn của bạn — check nào hiện tại bắt được?

1. **Hidden Technical Debt in Machine Learning Systems** — Sculley et al., Google, NeurIPS 2015. Vì sao phần lớn công sức của một hệ thống ML nằm ngoài model. Kinh điển, đọc 30 phút.
2. **“Everyone wants to do the model work, not the data work”: Data Cascades in High-Stakes AI** — Sambasivan et al., Google Research, CHI 2021. Nguồn của khái niệm **data cascade**. Nghiên cứu định tính, phỏng vấn 53 practitioner — đọc như một checklist chẩn đoán đáng tin, đừng đọc như số liệu thống kê dân số.
3. **Great Expectations 1.x Documentation** — *docs.greatexpectations.io*. Lưu ý API 1.x khác hẳn 0.x — `pin great-expectations==1.19.1`.
4. **Evidently Documentation** — *docs.evidentlyai.com*. Drift và distribution report; bản có team bảo trì, thay cho `drift.py` ở hoạt động B.
5. **dbt — Transform Data** — *docs.getdbt.com*. SQL transformation as code, data contract, version control cho logic data.

Hỏi & Đáp

Nếu chỉ được giữ đúng một dashboard cho hệ thống AI của mình — bạn giữ “chất lượng câu trả lời của agent” hay “data observability”?



Cảm ơn!

Ngày tiếp theo: Guardrails & AI Safety

labs + source code: github.com/vbi-academy/aicb-phase1